

# Kafka Order Platform

## 17 chapters. One connected story.

Simple English to speak • Hinglish cues to recall • Architecture + evidence

### YOUR CHAPTER MAP

|    |                                    |    |
|----|------------------------------------|----|
| 01 | Project Overview & Architecture    | 2  |
| 02 | Kafka Cluster, Topics & Partitions | 4  |
| 03 | Order Processing & Outbox          | 6  |
| 04 | Inventory, Idempotency & ACK       | 7  |
| 05 | Retry, DLT & Controlled Replay     | 8  |
| 06 | Avro & Schema Evolution            | 9  |
| 07 | Results & Order Lifecycle          | 10 |
| 08 | Monitoring, Metrics & Alerts       | 11 |
| 09 | Real Failures & Debugging          | 12 |
| 10 | Docker & Runtime Networking        | 13 |
| 11 | Jenkins CI: Test to Image          | 14 |
| 12 | Docker Hub & Image Versions        | 15 |
| 13 | CD, Approval & Health Checks       | 16 |
| 14 | GitHub Webhook & ngrok             | 17 |
| 15 | Secrets & Configuration            | 18 |
| 16 | Testing & Verification Evidence    | 19 |
| 17 | Demo & Interview Walkthrough       | 20 |

### PRACTISE THE ANSWER, THEN EXPLAIN THE REASON

Overview → core flow → failure handling → operations → evidence.

Use the short spoken answer, then discuss the implementation and its limits.

Personal project, production-readiness patterns. Company experience stays separate.  
Source checked 29 Sep 2026 · main @ 2bad80e88cf7 · no new runtime test claimed.

# Project Overview & Architecture

Simple English to speak | Hinglish cues to recall | Production-style design

## 01 PROJECT INTRODUCTION - SPEAK THIS ONCE

I built a Kafka-based order processing platform using Java 17 and Spring Boot. It has two services: Order and Inventory. The Order Service accepts requests and stores orders. The Inventory Service checks stock and sends the reservation result back. I used transactional outboxes and idempotent consumers to handle publication failures and duplicate delivery. I also added retry handling, a dead-letter topic, monitoring, and a Jenkins pipeline for local container deployment.

**Bolne ka order:** purpose > services > reliability > delivery.

## 02 CORE CONCEPTS - ONE DEFINITION EACH

**Event-driven:** Services react to events to perform their work.

Event aaya, tab next service ka kaam start hota hai.

**Loose coupling:** Services depend on event contracts rather than direct calls for this workflow.

Producer ko consumer ki immediate response ka wait nahi.

**Eventual consistency:** Related service states become consistent after events are processed.

Order status turant final nahi hota.

## 03 TECH STACK - EACH TOOL'S ROLE

|                                      |                                                           |
|--------------------------------------|-----------------------------------------------------------|
| <b>MySQL + JPA/Hibernate</b>         | Durable business data and local transactions.             |
| <b>Kafka + 3-node KRaft</b>          | Event transport; broker replication and metadata control. |
| <b>Avro + Schema Registry</b>        | Order-event schema and compatibility management.          |
| <b>Actuator + Micrometer</b>         | Application health and metrics exposure.                  |
| <b>Prometheus + Grafana</b>          | Inventory metrics collection, dashboards and alerts.      |
| <b>Docker + Jenkins + Docker Hub</b> | Container packaging, CI/CD and versioned images.          |

## 04 WHY KAFKA INSTEAD OF ONLY REST?

I wanted asynchronous processing so order acceptance would not wait for inventory processing. The trade-off is delayed final status and more failure-handling work.

# Project Overview & Architecture

Architecture walkthrough | Follow the arrows from request to final status



## 05 OTHER ROUTES - KNOW THE DIFFERENCE

**Duplicate event:** existing eventId > skip business update > ACK safely.

**Retryable technical failure:** initial attempt + 2 retries, 2-second backoff; if attempts still fail, successful DLT publication routes it to orders.created.avro-dlt.

Stock kam hona business rejection hai; technical failure ka DLT route alag hai.

## 06 BOUNDARIES - ANSWER WITHOUT OVERCLAIMING

**Delivery:** at-least-once; MySQL commit and Kafka offset commit are separate.

**Scope:** order + inventory only; no payment, shipping or live delivery tracking.

**Deployment:** local Docker/Jenkins; replay is a proof-of-concept.

Source: current repository README + OrderController + Inventory processing/consumer.

# Kafka Cluster, Topics & Partitions

Broker, partition, replication and KRaft | One clear answer per concept

## 01 THE WORDS YOU MUST BE ABLE TO EXPLAIN

|                           |                                                                           |
|---------------------------|---------------------------------------------------------------------------|
| <b>Broker</b>             | A Kafka server that stores partition replicas and serves client requests. |
| <b>Topic</b>              | A named stream of related records, divided into partitions.               |
| <b>Partition</b>          | An ordered log of records that enables parallel processing.               |
| <b>Offset</b>             | A record's position within one partition; it is not a global event ID.    |
| <b>Replication factor</b> | The total number of copies of each partition, including the leader.       |
| <b>ISR</b>                | In-sync replicas: the replicas Kafka considers caught up with the leader. |

## 02 PARTITIONS DISTRIBUTE DATA; REPLICAS COPY IT

Illustration only: 3 partitions, replication factor 3. L = leader; F = follower.  
Business-topic counts and leader placement must be checked at runtime.

| Partition | Broker 1 | Broker 2 | Broker 3 |
|-----------|----------|----------|----------|
| P0        | L: P0    | F: P0    | F: P0    |
| P1        | F: P1    | L: P1    | F: P1    |
| P2        | F: P2    | F: P2    | L: P2    |

**3 partitions x RF 3 = 9 partition replicas, not 9 different data streams.**

A producer writes to the partition leader; followers replicate its records.

## 03 ORDERING & PARALLELISM - FOLLOW-UP ANSWERS

**Ordering:** Kafka preserves log order within a partition, not across a topic.

**Record key:** with key-based partitioning and an unchanged partition count, the same key routes to the same partition.

**Scale:** more partitions allow more active consumers in one consumer group; extra consumers beyond the partition count stay idle in normal group assignment.

# Kafka Cluster, Topics & Partitions

Your actual local setup | Network paths | Failure and durability questions

## 04 KRAFT - METADATA CONTROL, WITHOUT ZOOKEEPER

KRaft manages Kafka metadata through a controller quorum.

**Our setup:** Kafka 4.3.1; all three nodes run broker + controller roles.

Three controllers need a majority of two; one controller can be unavailable.

### Windows-host applications

EXTERNAL: localhost + host ports

### Docker application containers

INTERNAL: broker names + 19092

**kop-kafka-1 | kop-kafka-2 | kop-kafka-3**

Broker data traffic: 19092 | Controller quorum: 19093

## 05 LISTENERS - WHERE TO CONNECT

**listeners:** addresses and ports where the broker accepts connections.

**advertised.listeners:** addresses Kafka returns for clients to connect to.

| Broker | Windows host   | Inside Docker     |
|--------|----------------|-------------------|
| 1      | localhost:9092 | kop-kafka-1:19092 |
| 2      | localhost:9094 | kop-kafka-2:19092 |
| 3      | localhost:9096 | kop-kafka-3:19092 |

**Why not localhost inside Docker?** It points to that container itself.

Bootstrap servers provide initial discovery; advertised addresses must also work.

## 06 DURABILITY - WHAT IF A BROKER FAILS?

**acks=all:** producer waits for all current ISR replicas to acknowledge.

**min.insync.replicas:** minimum ISR size required for an acks=all write.

**Example:** RF=3, min ISR=2; writes can continue with two in-sync replicas.

If ISR falls to one, writes are rejected. Leader loss needs an eligible replacement.

**Verified vs assumed:** Compose sets internal offsets/transaction-topic RF=3 and transaction-topic min ISR=2. These do not set business-topic values. Order producer uses acks=all. Business-topic RF/min ISR need a runtime check. All containers share one laptop: this is not protection from laptop failure.

**Interview scope:** combined broker/controller roles suit this local setup; critical deployments should separate roles. Consumer ACK is a different concept from producer acks; we will cover it in the consumer chapter.

Sources: repository docker-compose.yml + README; Apache Kafka KRaft and topic configs.

# Order Processing & Outbox

One local transaction • Reliable publication • The duplicate window

## SAY IT IN THE INTERVIEW

“I save the order and its event in the same MySQL transaction. A scheduled publisher sends pending events to Kafka. I mark a row PUBLISHED only after a successful send. This protects the order-to-event handoff, but duplicates are still possible, so consumers must be idempotent.”

## 01 WHY THIS PATTERN EXISTS

- **Transactional Outbox:** save business data and an event-to-publish in the same database transaction; publish it separately.
- Direct DB save + Kafka send is a **dual write**. One can succeed while the other fails. Our local transaction covers **orders + outbox**; it does not include Kafka.

## 02 WHAT MY CODE DOES

- **POST /api/orders:** create order with status **pending**; create ORDER\_CREATED outbox row with status **PENDING**. @Transactional commits both or rolls back both.
- Publisher polls pending rows by created time; **fixedDelay=1000** means a delay after the previous run finishes, not a guaranteed event every second.
- Stored payload is JSON. Publisher converts it to Avro, sets **source=WEB**, and sends to **orders.created.avro** using **orderId** as the Kafka key.

## 03 CRASH WINDOWS — THE IMPORTANT FOLLOW-UP

- **Before DB commit:** neither durable order nor durable outbox entry.
- **After DB commit, before send:** row remains PENDING; the publisher can retry.
- **Kafka accepts, process dies before PUBLISHED save:** the same event can be sent again. Preserve its eventId; see Chapter 04 for consumer protection.
- Successful send → save PUBLISHED + publishedAt. A failed send is not treated as successful publication.

## 04 TIMEOUTS & HONEST LIMITS

- Registry HTTP connect/read: **60s**; Kafka delivery: **120s**; publisher future wait: **130s**. The future wait starts after publish() returns; synchronous serialization can add time before it.
- Failure/timeout keeps the durable row pending; interruption also restores the thread interrupt flag. Timeout can mean an **unknown send outcome**, so retries may duplicate.
- Current publisher has no row-claiming lease for multiple instances. This is polling outbox, not CDC; published-row retention is a future operational concern.

**YAAD RAKHO: DB commit ≠ Kafka publish; Kafka publish ≠ outbox status save.**

# Inventory, Idempotency & ACK

Commit business work first • Survive redelivery • Protect concurrent stock updates

## SAY IT IN THE INTERVIEW

“Kafka can redeliver a record after the database has committed. I use `eventId` to detect an already processed event. Stock changes, the processed-event record and the result outbox are in one transaction. The listener acknowledges only after that transaction succeeds, including the duplicate-skip path.”

## 01 ONE TRANSACTION, THREE RELATED WRITES

- **Idempotency:** processing the same event again must not repeat its business effect. Our key is **`eventId`**, not the Kafka offset or `orderId`.
- For a new event: load inventory → save `ProcessedEvent` → reserve stock or record rejection → save result outbox. A technical exception rolls back this transaction.
- **`processed_events.event_id`** has a unique constraint. The existence check helps normal duplicates; the database constraint protects concurrent duplicate inserts.

## 02 ACK BELONGS AFTER THE TRANSACTION

- Consumer calls the separate **`@Transactional`** service. The Spring proxy commits before control returns to the listener; then `acknowledge()` runs.
- **Manual ACK** tells the listener container that processing is complete so it can commit the consumed position. It does not delete the Kafka record.
- Crash after DB commit but before offset commit? Redelivery finds `eventId`, skips business work, and reaches ACK. ACK before DB commit could lose work after a crash.
- Technical failure escapes the service, so the normal ACK line is not reached. Retry/DLT behavior is covered in Chapter 05.

## 03 TWO DIFFERENT STOCK CONCERNS

- **Insufficient stock:** a valid business outcome. Commit the rejection result + processed marker; acknowledge it. It is not a poison event.
- **Optimistic locking:** `@Version` detects another transaction changing the same inventory row before our update commits. A stale update fails instead of silently overwriting.
- Stock is changed on a managed JPA entity; dirty checking persists it on commit. Kafka is keyed by **`orderId`**, so different orders for one product can still contend.

## 04 FOLLOW-UP: IS THIS EXACTLY-ONCE?

- “I use at-least-once delivery with idempotent database processing. I do not claim Kafka exactly-once semantics or one distributed Kafka/MySQL transaction.”
- `eventId` must stay stable on replay. A new `eventId` looks like a new business event; retention of processed IDs must consider possible late redelivery.

**YAAD RAKHO:** same `eventId` → skip effect; successful DB commit → ACK.

# Retry, DLT & Controlled Replay

Separate business rejection from technical failure • Recover deliberately

## SAY IT IN THE INTERVIEW

“For a retryable processing failure, I allow the initial attempt and two retries with a two-second fixed backoff. If processing still fails, the error handler publishes the record to the DLT. I inspect the failed record, fix the cause and use a targeted replay proof-of-concept.”

## 01 WHICH PATH SHOULD THE RECORD TAKE?

- **Business rejection:** insufficient stock produces `inventory.reservation.failed` and commits normally. Retrying unchanged stock is not our business policy.
- **Technical failure:** an exception leaves the normal ACK path. `DefaultErrorHandler` uses `FixedBackOff(2000L, 2L)`. This is the configured retryable path; do not claim every exception always gets three attempts.

## 02 DLT RECOVERY HAS TO SUCCEED

- **Dead Letter Topic:** a separate topic for records that cannot be processed on the normal path. Here: `orders.created.avro-dlt`.
- `DeadLetterPublishingRecoverer` selects the original partition number. Provision the DLT with enough partitions for that routing.
- `failIfSendResultIsError=true` surfaces DLT send failure. `commitRecovered=true` with `MANUAL_IMMEDIATE` allows the recovered offset to advance after successful recovery.

## 03 INSPECTION IS DIFFERENT FROM REPLAY

- Inspection group: `inventory-dlt-inspection-group`. It receives raw bytes, tries Avro decoding and logs the record.
- The inspector ACKs even after a decode failure or null value. This advances its own inspection position; it neither deletes the record nor repairs business data.
- Before replay: identify `eventId/orderId`, inspect and fix the cause. Preserve event identity for the duplicate protection in Chapter 04.

## 04 WHAT THE CURRENT REPLAY RUNNER REALLY DOES

- Enabled only by `app.dlt.replay.enabled=true`; disabled by default. Scans DLT partition **1** from the beginning, matching one hard-coded `orderId + eventId`.
- A 30-second scan finds the target; its send waits up to 30 seconds. It republishes to the source topic, then returns from the runner.
- No general replay API, durable replay audit or rate control. Legacy scripts use `orders.created-dlt`, not the current Avro route.

**YAAD RAKHO:** reject = business result; DLT = investigate; replay = controlled retry.

SOURCE CHECK: `KafkaErrorHandlingConfig` + `InventoryDltConsumer` + `InventoryDltReplayRunner`

# Avro & Schema Evolution

A shared event contract • Generated Java types • Safe change boundaries

## SAY IT IN THE INTERVIEW

"I use Avro for OrderCreated events and Schema Registry to manage their schemas. I added a source field with the default UNKNOWN, while new events set WEB. That lets a new reader resolve older records that do not contain source. Inventory result events currently remain JSON."

## 01 WHAT EACH PIECE DOES

- **Avro:** schema-based binary serialization. **Schema Registry:** stores schema versions and supports compatibility checks; Kafka still stores the event records.
- The Confluent Avro serializer writes a schema identifier with encoded data, not the full schema in every record. The consumer resolves the writer schema to decode it.
- Both services have semantically matching **order-created.avsc** files. avro-maven-plugin generates Java classes during **generate-sources**. Do not edit generated classes.

## 02 OUR ACTUAL EVENT CONTRACT

- Identity: **eventId**, **orderId**. Business data: **productId**, **quantity**, **amount**, **status**. Metadata: **occurredAt**, **source**.
- amount uses decimal with precision **19**, scale **2**; occurredAt uses timestamp-millis. These logical types preserve business meaning beyond raw bytes/longs.
- **orders.created.avro** uses Avro. **inventory.reserved** and **inventory.reservation.failed** use JSON. Do not describe every topic as Avro.

## 03 EXPLAIN THE CHANGE WITH ONE EXAMPLE

- Old writer: no source field. New reader: source string with default **UNKNOWN**. While reading that older record, the reader supplies UNKNOWN.
- New publisher explicitly sets **WEB**. A reader default supports schema resolution; it does not retroactively rewrite old Kafka records.
- **Backward compatibility:** a new reader can read old data. Forward is the reverse; full requires both directions. Actual Registry subject compatibility must be checked, not assumed from the file alone.

## 04 FOLLOW-UP QUESTIONS I SHOULD HANDLE

- **Why Avro over plain JSON?** An explicit contract, generated types and controlled evolution. Trade-off: schema tooling and Registry availability become operational dependencies.
- **Registry is down?** A needed registration/schema lookup can fail or block; cached schemas may allow some operations. It is not correct to say all sends always fail immediately.
- **Safe change?** Regenerate/build both services, check compatibility and test old/new records. Defaults do not make every change safe.

**YAAD RAKHO:** old record + new reader → source UNKNOWN; new event → WEB.

SOURCE CHECK: Both order-created.avsc files + Maven plugins; Avro/Confluent specifications

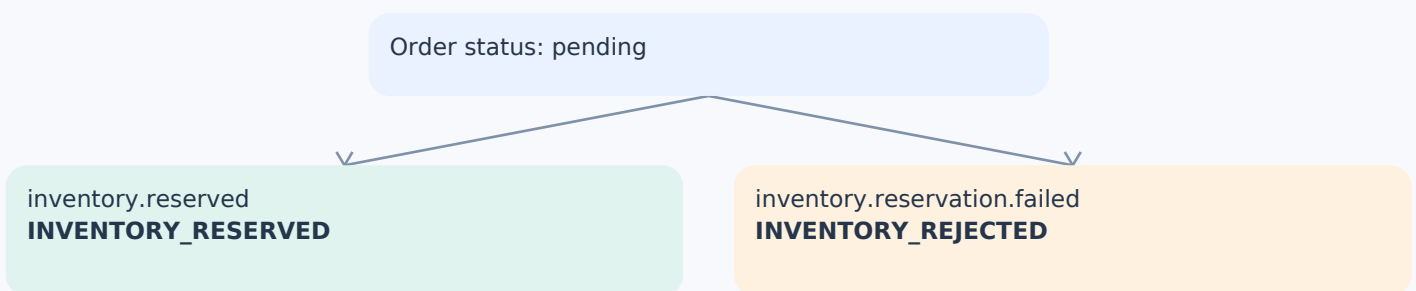
# Results & Order Lifecycle

Close the asynchronous loop • Correlate events • Understand state boundaries

## SAY IT IN THE INTERVIEW

“Inventory publishes either a reserved or a rejected result through its own outbox. Order Service consumes that result and updates the order in a transaction with a processed-event record. The initial HTTP response only accepts the order; the final inventory outcome arrives asynchronously.”

## 01 TWO BUSINESS OUTCOMES



## 02 IDENTITY & TRANSACTION BOUNDARIES

- **eventId:** identifies this result event. **sourceEventId:** links it to the original OrderCreated event.
- **orderId:** identifies the business order.
- Inventory transaction creates the result outbox alongside its business work. Its publisher sends the corresponding JSON event using orderId as key.
- Order's markInventoryReserved(...) and markInventoryRejected(...) are both **@Transactional:** status update + processed marker commit together. The listeners ACK afterwards.

## 03 WHAT A SLOW RESULT MEANS

- If the result outbox is still PENDING, Inventory may already have committed its decision while Order still shows pending. Trace the result publisher and Order consumer before retrying the client request.
- The Inventory publisher calls **send(...).get()** without an explicit future-wait timeout. It does not yet have the Order publisher's interruption hardening.

## 04 DO NOT OVERSTATE THE WORKFLOW

- Duplicate protection is covered in Chapter 04; it does not validate every possible state transition. Current setters do not enforce a complete lifecycle state machine.
- Two result topics do not provide a shared global order. Distinct conflicting result events need stronger transition/version rules if introduced.
- This is an inventory decision workflow, not a complete payment/shipping saga. There is no compensation engine or guaranteed end-to-end completion deadline.

**YAAD RAKHO: HTTP accepted  $\neq$  inventory decision; result event closes the loop.**

SOURCE CHECK: Inventory result publisher + Order result consumers + OrderApplicationService

# Monitoring, Metrics & Alerts

Read the signal correctly • Explain the outage exercise • Know current coverage

## SAY IT IN THE INTERVIEW

"I exposed health and Prometheus metrics through Actuator. Prometheus scrapes Inventory Service every five seconds, and Grafana uses those metrics for monitoring and service-down alerts. I exercised the alert behavior locally. The current Compose configuration keeps SMTP disabled unless I explicitly configure it."

## 01 WHO DOES WHAT?

- **Actuator** exposes operational endpoints. **Micrometer** provides application/JVM meters. **Prometheus** collects time-series samples. **Grafana** queries and visualizes them, and evaluates configured alerts.
- Both services expose **/actuator/health** and **/actuator/prometheus**. The committed scrape configuration targets only **Inventory :8082** every **5s**.

## 02 THE AVAILABILITY SIGNAL

- **up=1**: the latest target scrape succeeded. **up=0**: it failed. A failed scrape can mean the app, network or metrics endpoint is unavailable.
- Example query: **up{job="inventory-service"}**. Use the same job label when tracing the alert condition.
- A service-stop exercise should show failed scrapes, a firing alert after its configured evaluation/pending period, then recovery after successful scrapes resume. Five-second scraping is not a guaranteed five-second notification.

## 03 WHAT I CAN SHOW FROM THIS PROJECT

- Committed captures include Prometheus target-up, a Grafana dashboard folder and separate Firing/Normal alert states. They support the local monitoring walkthrough.
- The dashboard image shows the folder listing, not panel graphs. Separate alert-rule identifiers are not proof of one correlated firing-to-recovery history.
- Grafana UI configuration persists in **grafana-data**. Dashboard/alert provisioning files are not committed; a fresh machine needs that setup.

## 04 HOW I ANSWER AN OPERATIONS FOLLOW-UP

- "I check health, scrape status and logs first, then trace the affected event through the databases and Kafka flow. A healthy process can still have a stuck business workflow."
- **Future additions**: Order-service scraping, outbox backlog/age, DLT rate and consumer-lag monitoring. These are useful next signals, not claimed current dashboards.
- Alert evaluation and email delivery are separate. **GF\_SMTP\_ENABLED=false** is the default; an email contact-point label does not prove message delivery.

**YAAD RAKHO: process health, scrape health and business completion are different.**

# Real Failures & Debugging

Use observed incidents • Explain your reasoning • Avoid invented production claims

## SAY IT IN THE INTERVIEW

“One difficult issue in my local project was an outbox publisher that appeared stuck. I traced the blocking path to Avro schema registration over HTTP, before Kafka returned the send future. I adjusted the timeout layers, added correlated logs and verified the business flow after recovery.”

## 01 STORY 1 — THE OUTBOX THAT LOOKED HUNG

- **Situation:** an order existed but publication was delayed. I followed `eventId/orderId` through `OutboxPublisher` → `OrderEventProducer` → `KafkaAvroSerializer` → Registry registration.
- **Evidence:** the documented local run observed one Registry request taking about **26.9 seconds**. A temporary 10-second HTTP limit was too aggressive for that run.
- **Action:** timeout settings and interruption handling were hardened (Chapter 03). **Result:** the recorded verification reached `PUBLISHED` and `INVENTORY_RESERVED`, with no pending Order outbox rows from that run.

## 02 STORY 2 — CONTAINER STARTED, DEPLOYMENT FAILED

- **Situation:** starting a Docker container did not establish readiness. The earlier curl retry did not necessarily retry the complete health predicate.
- **Action:** retry curl success **and** the UP-content check together. **Result:** the Jenkinsfile now contains the full-condition loop; Chapter 13 explains its exact boundary.
- **Lesson:** distinguish process creation from application readiness. Do not invent an outage duration or a performance improvement percentage.

## 03 STORY 3 — LOCAL RESOURCE PRESSURE

- **Situation:** the documented 8 GB development laptop showed pressure while running the full stack, alongside slow startup and intermittent failures.
- **Recovery:** reduce competing load, recover brokers, then start Registry after broker health. Compose includes broker-health dependencies.
- **Lesson:** local resource pressure was an observed contributor, not a proven explanation for every broker timeout or a Kafka architecture defect.

## 04 A SIMPLE ANSWER STRUCTURE

- Use **symptom** → **evidence** → **change** → **verification** → **remaining limit**. State which part you personally implemented or debugged.
- Say **“In my personal hands-on project...”** for these stories. Company project, scale, production traffic and team ownership require your actual work history.

**YAAD RAKHO:** symptom → evidence → action → verified result → lesson.

# Docker & Runtime Networking

Know what Compose starts • Know what Jenkins deploys • Trace connection paths

## SAY IT IN THE INTERVIEW

"I use Docker Compose for the Kafka cluster, Schema Registry and monitoring. Jenkins deploys the two application containers separately onto the Compose network. MySQL remains on the Windows host, so the application containers reach it through `host.docker.internal`."

## 01 IMAGE, CONTAINER & OUR BUILD

- **Image:** a packaged runtime filesystem and startup configuration. **Container:** a running instance created from that image.
- Both service Dockerfiles use **eclipse-temurin:17-jre**, copy `target/*.jar` to `/app/app.jar` and start **`java -jar app.jar`**. Maven builds the JAR before docker build.
- This is a simple runtime image, not a multi-stage Maven build. The custom Jenkins image is based on `lts-jdk21` and installs Docker tooling.

## 02 WHO OWNS WHICH RUNTIME?

- **Compose:** `kop-kafka-1/2/3`, `kop-schema-registry`, `kop-prometheus` and `kop-grafana`.
- **Jenkins deployment:** `order-service-container` on **8080** and `inventory-service-container` on **8082**. MySQL is not a Compose service.
- Application network: **`kafka-order-platform_default`**. Its existence depends on the Compose project name; running from another project name can change the default network.

## 03 THREE CONNECTION RULES

- Container → Kafka: **`kop-kafka-1:19092`** (and peers). Container → Registry: **`kop-schema-registry:8081`**. See Chapter 02 for the full broker-port map.
- Container → host MySQL: **`host.docker.internal:3306`**. Inside a container, `localhost` refers to that container; it is not the Windows host.
- Host → application uses a published port, such as **`8080:8080`**. Shared-network container communication uses service/container names and internal ports.

## 04 PERSISTENCE & STARTUP

- Each Kafka broker has its own named data volume. Grafana has **`grafana-data`**. Volumes can survive container replacement; deleting a volume deletes its persisted data.
- Prometheus configuration is a read-only bind mount; no explicit Prometheus data volume is declared in current Compose. Do not assume equivalent metrics-history durability.
- Registry depends on all three broker health checks. This controls startup ordering, not continuous dependency recovery or business readiness.

**YAAD RAKHO: Compose = infrastructure; Jenkins = apps; Windows host = MySQL.**

# Jenkins CI: Test to Image

Reproducible steps • Test gates • Understand exactly what each artifact contains

## SAY IT IN THE INTERVIEW

“My Jenkins pipeline checks out main, runs each service's tests, packages and archives its JAR, then builds both Docker images. Test stages run before packaging. I use skipTests during packaging to avoid running the same tests twice, not to bypass the earlier test gates.”

## 01 THE ACTUAL STAGE ORDER

- **Checkout main** → Order test → Order package → Order JAR archive → Inventory test → Inventory package → Inventory JAR archive → build both Docker images.
- These stages are sequential in the current Jenkinsfile. A failed required test command stops later normal stages; image publication follows the successful CI path.
- **Continuous integration:** automatically build and verify integrated changes. CD and the manual deployment decision are covered in Chapter 13.

## 02 TEST ENVIRONMENT MATTERS

- Order: **./mvnw test** with a datasource URL override to host MySQL **order\_db**. This is not a dedicated Order test database.
- Inventory: **./mvnw test** with **TEST\_DB\_URL** pointing to **inventory\_test\_db** and **TEST\_SCHEMA\_REGISTRY\_URL** pointing to host port **8081**.
- Embedded Kafka supplies the broker for the DLT integration test; MySQL and Schema Registry remain external dependencies. “Embedded” does not mean the whole test environment is self-contained.

## 03 JAR AND IMAGE ARE DIFFERENT OUTPUTS

- **./mvnw package -DskipTests** packages each service. Jenkins archives **service/target/\*.jar** for download from the build.
- docker build uses each service directory and Dockerfile. Local tags are **order-service:\${BUILD\_NUMBER}** and **inventory-service:\${BUILD\_NUMBER}**.
- A JAR archive does not publish a Docker image. An image build does not push to a registry. Push stages are separate (Chapter 12).

## 04 WHEN A BUILD FAILS, WHAT DO I CHECK?

- First identify the failing stage. For tests, inspect the Maven failure and DB/Registry reachability. For image builds, check that packaging produced the expected JAR.
- Use the checked-out **Git SHA + Jenkins build number** to identify the source and build. A moving main branch name alone is insufficient.
- Current Jenkinsfile archives JARs but does not publish JUnit XML through a junit step. The tests exist; a coverage percentage is not measured by counting test methods.

**YAAD RAKHO: test gate → package JAR → archive JAR → build image → push later.**

SOURCE CHECK: Jenkinsfile test/package/archive/build stages + service test configurations

# Docker Hub & Image Versions

Publish a specific build • Track the source • Separate publication from deployment

## SAY IT IN THE INTERVIEW

“Jenkins tags both service images with BUILD\_NUMBER and pushes them to Docker Hub. The Docker Hub token is stored in Jenkins Credentials and passed through standard input during login. I can trace a numbered image back to its Jenkins build and the source commit recorded by that build.”

## 01 THE NAMES I ACTUALLY USE

- **Local:** order-service:\${BUILD\_NUMBER}
- Registry:** saifee162007/order-service:\${BUILD\_NUMBER}
- **Local:** inventory-service:\${BUILD\_NUMBER}
- Registry:** saifee162007/inventory-service:\${BUILD\_NUMBER}
- Example: build **28** produces both **:28** tags. The current pipeline does not also push latest.

## 02 BUILD, TAG, PUSH — THREE DIFFERENT ACTIONS

- **docker build** creates an image. **docker tag** adds another name to an existing image; it does not rebuild it. **docker push** publishes the registry-qualified image.
- Both services use the same build number for correlation, but remain separate images. One successful push does not make the other push atomic.
- A Docker Hub tag can exist even if later approval/deployment fails. The committed captures show tags 28 and 30; they are publication evidence, not complete pipeline-success evidence.

## 03 AUTHENTICATION WITHOUT EMBEDDING A TOKEN

- Credential ID: **dockerhub-credentials**. Jenkins withCredentials binds the username and token for the login step.
- **docker login --password-stdin** receives the token through stdin rather than a password argument. The Jenkinsfile contains the credential reference, not the actual token.
- Jenkins masking is useful, but not permission to print secrets. Docker login can retain authentication in the agent's Docker configuration; agent access still matters.

## 04 VERSIONING TRADE-OFFS

- A numbered tag makes a build easier to identify than latest. It is not automatically immutable: a registry tag can be overwritten unless policy prevents it.
- Current tags do not embed a Git SHA or semantic version. Retain the Jenkins build record to connect **Git commit** → **build** → **image tag**.
- Current deployment runs the already-built **local image** after push; it does not pull it back from Docker Hub.

**YAAD RAKHO:** tag identifies a release candidate; successful push ≠ successful deploy.

# CD, Approval & Health Checks

Deploy deliberately • Verify readiness • Explain failure after container replacement

## SAY IT IN THE INTERVIEW

“After publishing both images, Jenkins waits for manual deployment approval. It replaces Order and Inventory containers, then checks both Actuator health endpoints. This is a continuous-delivery workflow with an approval gate. The current local setup does not provide automatic rollback or zero-downtime deployment.”

## 01 WHAT HAPPENS AFTER APPROVAL?

- **Before approval:** images are prepared; the deployment stages have not replaced running apps. The Jenkins input step offers the Deploy decision.
- **After approval:** remove old Order container → start new Order → remove old Inventory → start new Inventory → check Order health → check Inventory health.
- Runtime environment variables supply the Docker/host connection addresses described in Chapter 10. The same application code runs with different endpoint configuration.

## 02 THE READINESS PREDICATE

- For each service: **up to 30 attempts**, with **3-second waits** between failures. The loop reruns curl against **/actuator/health** and checks for **"status":"UP"**.
- Jenkins reaches Order at host.docker.internal:8080 and Inventory at host.docker.internal:8082. curl uses --fail; the response must also satisfy the text check.
- This is up to **29 sleeps** plus HTTP request time, not a strict 90-second deadline. The current curl command has no explicit per-request timeout.

## 03 WHAT FAILURE LEAVES BEHIND

- If the final check fails, Jenkins exits with an error. It does not automatically restore the old image or container.
- Old containers are removed before the new versions are proven healthy. There can be downtime; a failure between deployments can leave mixed service versions.
- Both replacements happen before health verification. A successful Order start is not a gate that verifies Order health before Inventory replacement.

## 04 HOW I WOULD IMPROVE IT

- **Next:** explicit request deadlines, structured top-level health parsing, retained previous versions, rollback validation and an end-to-end smoke order.
- For stronger availability, evaluate staged/blue-green deployment. Explain the database and event-contract compatibility required during mixed-version operation.
- Actuator UP is a readiness signal in this pipeline; it does not prove an order completed correctly. Use the correlated demo in Chapter 17 for that.

**YAAD RAKHO:** approval → replace → health check; failure currently needs recovery.

# GitHub Webhook & ngrok

A push can trigger Jenkins • A public route reaches a local endpoint

## SAY IT IN THE INTERVIEW

“My Jenkins server runs locally on port 8085. I used ngrok to expose a public HTTPS route, then configured GitHub to send push events to Jenkins through that route. I verified automatic pipeline triggering using empty commits, without changing the application code.”

## 01 THE THREE ROLES

- **Webhook:** GitHub sends an event notification after a push. It does not build or deploy the application itself.
- **ngrok:** forwards the public HTTPS request to local Jenkins. GitHub cannot directly call my machine's localhost.
- **Jenkins:** receives `/github-webhook/`, checks the configured repository and runs the job. The job checks out main and executes its Jenkinsfile.

## 02 WHAT I CONFIGURED OUTSIDE GIT

- Jenkins UI: **GitHub hook trigger for SCM polling**. GitHub repository settings: the public webhook URL ending with `/github-webhook/`.
- The active tunnel, hostname, GitHub webhook settings and Jenkins trigger checkbox are runtime/UI configuration, not recreated by cloning the repository.
- A tunnel URL may change between sessions. A stale GitHub webhook URL can break the trigger while the application and Jenkinsfile remain correct.

## 03 HOW I TESTED AND HOW I WOULD DEBUG

- Recorded test commits: **13e0a345** (“Test Jenkins webhook”) and **de9f5e97** (“Verify Jenkins webhook”). An empty commit generates a real push without source changes.
- The project history reports a successful callback and automatic job start. A commit alone proves a push attempt, not successful webhook delivery.
- Troubleshoot in order: GitHub delivery response → active tunnel/forwarding port → Jenkins reachability → trigger setting/repository match → job/build cause.
- Redelivery sends the existing webhook again; an empty commit creates a new Git event. Use these to isolate trigger issues from application failures.

## 04 KEEP THESE CLAIMS PRECISE

- The webhook triggers the pipeline; it does not bypass the manual deployment approval described in Chapter 13.
- The public tunnel exposes a route to local Jenkins. Keep tokens private; do not claim webhook-signature validation is configured without checking the actual settings.

**YAAD RAKHO: webhook notifies; ngrok forwards; Jenkins builds; approval gates deploy.**

# Secrets & Configuration

Separate non-secret settings from credentials • Explain the known migration gap

## SAY IT IN THE INTERVIEW

"I keep environment-specific endpoints separate from Java code and override them for Docker deployment. Docker Hub authentication uses Jenkins Credentials. I also identified a remaining issue: database passwords are still in tracked Spring properties. That needs rotation and externalization; I do not present it as already fixed."

## 01 WHICH SETTING GOES WHERE?

- **Repository:** topic names, ports, retry policy and safe default addresses. These describe behavior and are not authentication secrets.
- **Runtime environment:** values such as `SPRING_DATASOURCE_URL` and `SPRING_KAFKA_BOOTSTRAP_SERVERS` adapt the app to the host or Docker network.
- **Secret store/local override:** actual passwords and tokens. A Jenkins credential ID is a reference, not the secret value. See Chapter 12 for the implemented Docker Hub binding.

## 02 TWO EASY CONFIGURATION MISTAKES

- An ignored `.env` is not automatically read by Spring Boot. Compose interpolation only helps when the Compose file references the relevant variables.
- An ignored `application-local.properties` file needs the corresponding `local` profile activated. Ignoring a file does not make Spring load it.
- Current Compose has SMTP disabled and does not wire the optional SMTP placeholders into Grafana. Copying `.env.example` alone does not enable email.

## 03 THE DATABASE GAP — EXPLAIN THE SAFE FIX

- Tracked main/test Spring properties currently contain DB passwords; the deployment does not inject an external DB password. Never display their values in notes or demos.
- Migration plan: rotate the credential → configure Jenkins Credentials → externalize the Spring setting → pass `SPRING_DATASOURCE_PASSWORD` to tests/deployment → verify both services.
- Configure an ignored local override for development. Verify Jenkins has the new credential before removing its current dependency, so the pipeline is not silently broken.

## 04 WHAT IGNORE RULES DO NOT SOLVE

- The repository ignores `.env` and local Spring override files. This reduces accidental future commits; it does not untrack existing files or erase prior Git history.
- Deleting a committed password from the latest file is not rotation. Treat the old credential as exposed and replace it.
- ngrok tokens, webhook secrets and SMTP credentials must stay out of screenshots, logs and Git. Kafka uses PLAINTEXT here; TLS/ACL hardening is not a current guarantee.

**YAAD RAKHO:** ignored ≠ loaded; removed from latest file ≠ secret rotated.

# Testing & Verification Evidence

Know the assertions • Separate unit and integration scope • Avoid coverage claims

## SAY IT IN THE INTERVIEW

“The current repository has thirteen JUnit test methods. I use unit tests for business decisions and outbox failure handling, a MySQL integration test for persistence, and an Embedded Kafka test for the DLT route. I also demonstrated the end-to-end workflow manually; that is separate from automated coverage.”

## 01 THE VERIFIED TEST INVENTORY — 13 METHODS

- **Order context:** 1 startup test. **Order OutboxPublisher:** 6 tests.
- **Inventory context:** 1 startup test. **InventoryProcessingService:** 3 unit tests.
- **InventoryDatabaseIntegrationTest:** 1 persistence test.
- InventoryRetryToDLTIntegrationTests:** 1 Kafka/DLT integration test.

## 02 WHAT THE ASSERTIONS ACTUALLY PROTECT

- **Order outbox:** publish status only after successful send; timeout then retry; asynchronous failure; synchronous serialization failure; interruption preserved and batch stopped; empty poll.
- **Inventory unit:** duplicate skipped, new event reduces stock, insufficient stock creates a rejection outbox with expected payload.
- **MySQL test:** stock changes from **10 to 8** and the processed event exists. This checks actual persistence, not only mocked repository calls.
- **DLT test:** missing product triggers recovery; assertions check the key, selected decoded Avro fields, original-topic header and exception metadata. It does not explicitly count listener attempts.

## 03 ENVIRONMENT & SAFETY BOUNDARIES

- The DLT test embeds Kafka, not Schema Registry or MySQL. Jenkins supplies their URLs; the exact test commands are in Chapter 11.
- Inventory DB integration uses **create-drop**. Keep it on **inventory\_test\_db**, never a database whose data you need.

## 04 RUNTIME PROOF AND NEXT TESTS

- The repo has **16 screenshots**: monitoring, Jenkins, Docker, Docker Hub and MySQL. A missing additional capture does not mean you never performed the hands-on exercise.
- For a strong walkthrough, correlate one orderId across API, outboxes, stock and final status. Historical screenshots from different runs cannot establish that full sequence.
- Future automation: Order result-consumer tests, Inventory outbox failure tests, full end-to-end flow and deployment rollback verification. Existing tests remain useful without claiming those gaps are covered.

**YAAD RAKHO: test count  $\neq$  coverage; screenshot  $\neq$  automated assertion.**

# Demo & Interview Walkthrough

Start with your contribution • Show one correlated order • Finish with one real lesson

## SAY IT IN THE INTERVIEW

"I will first explain the order flow, then show one successful reservation and how I handle duplicates or failures. After that, I can walk through the Jenkins deployment and monitoring. This project helped me practise reliable event processing, debugging and production-readiness patterns in a local environment."

## 01 START THE EXISTING PROJECT IN THE RIGHT ORDER

- Use **D:\kafka-order-platform**; do not recreate it. Start host MySQL, then **docker compose up -d**; verify brokers and Registry readiness.
- Use one app mode: host processes **or** Jenkins-deployed containers. For host mode, run **.\mvnw.cmd spring-boot:run** in each service folder with local DB credentials supplied securely.
- Start Inventory, ensure the demo product exists with enough stock, then start Order. Check health on **:8082** and **:8080**. Keep replay disabled for normal startup.

## 02 A THREE-MINUTE BUSINESS DEMO

- **Before:** note source commit, timestamp and product stock. Submit **POST /api/orders** with productId, quantity and amount; retain the returned orderId.
- **Response:** HTTP **202** with a text orderId. Show the Order outbox, then Inventory processed event, stock and result outbox, then final Order status.
- With sufficient stock, expect **INVENTORY\_RESERVED** and the requested deduction. Correlate orderId across services; source-event and result-event IDs are different.
- Repeat HTTP submission creates another order. A duplicate-event demonstration must redeliver the same eventId and show no second stock deduction.

## 03 AN INTERVIEW ANSWER YOU CAN CONTROL

- **First minute:** use Chapter 01's introduction and architecture. **Next:** explain the transaction boundaries in Chapters 03–04. Then let the interviewer choose a deeper topic.
- For "a challenge you solved," use one Chapter 09 incident: symptom, evidence, fix and verification. Avoid reciting every tool before explaining the business problem.
- Useful English: **"Let me explain with one example."** / **"In my implementation..."** / **"I have not implemented that yet; my next step would be..."**

## 04 END WITH EVIDENCE AND A CLEAR BOUNDARY

- Do not guess throughput, user count or production ownership. These notes cover the personal project; introduce real company work separately using verified resume facts.
- For full commands, topic setup and safe shutdown, use the project README's **How to Run / Local Setup** chapter. Stop normally without deleting volumes.

**YAAD RAKHO:** explain one flow → show one order → discuss one failure → own the limits.

SOURCE CHECK: OrderController + README local setup + verified project workflow